

TriFusion — Trimodal Emotional Intelligence System

Complete Project Documentation | Production-Grade AI Internship Flagship Project

Table of Contents

1. [Project Overview](#)
 2. [Problem Statement](#)
 3. [Why This Project Stands Out](#)
 4. [System Architecture](#)
 5. [Tech Stack \(Full Breakdown\)](#)
 6. [Datasets & Models](#)
 7. [Module-by-Module Breakdown](#)
 8. [Agentic Workflow — WellnessAgent](#)
 9. [Streamlit UI — Pages & Preview Requirements](#)
 10. [Folder Structure](#)
 11. [Setup & Installation](#)
 12. [Evaluation Metrics](#)
 13. [Resume Bullets](#)
 14. [Shipping Checklist](#)
-

1. Project Overview

TriFusion is a real-time, trimodal emotional intelligence system that simultaneously analyzes a user's **facial expressions (vision)**, **voice tone (audio)**, and **spoken words (text)** to detect their true emotional state — including when those three signals *contradict each other* (emotional masking/incongruence).

On top of this detection engine sits **WellnessAgent** — a LangGraph-powered agentic AI that responds to the user's detected emotional state with personalized, real-time wellness interventions: breathing exercises, grounding techniques, affirmations, or human escalation — all driven by the fusion of three deep learning models working simultaneously.

One-line pitch for recruiters:

"A production-grade trimodal AI that reads your face, voice, and words simultaneously — detecting emotional masking using KL-divergence incongruence scoring — and deploys a LangGraph wellness agent that responds to your real emotional state, not just what you say."

2. Problem Statement

The Gap in Current Emotion AI

Today's emotion AI is **unimodal and naive**. A chatbot reads your text. A sentiment analyzer reads your words. Nobody builds systems that understand what humans have always known intuitively: **people lie with their words but rarely with their face and voice simultaneously**.

The Real-World Problem

Mental health and stress detection is one of the most critical unsolved problems in human-computer interaction:

- 75% of people experiencing acute stress report feeling "fine" when asked directly (APA, 2023)
- Crisis intervention systems miss early distress signals because they rely on self-reporting
- Customer service AI cannot detect frustrated users until they escalate verbally — losing trust and retention

What TriFusion Solves

TriFusion detects **emotional incongruence** — the gap between what a person says, how they sound, and what their face shows. This gap is precisely where masked distress, suppressed anxiety, and hidden frustration live.

Concrete example: A user says "I'm fine, just a bit tired" (Text → Positive/Neutral) while their voice is clipped and rapid (Audio → Stress) and their face shows corrugator activation and compressed lips (Vision → Suppressed Anger). Incongruence score: 0.87. WellnessAgent activates calm intervention protocol.

This is **not a toy demo**. This is the kind of system that mental health platforms, HR tech companies, customer support AI, and educational technology companies are actively trying to build.

3 Why This Project Stands Out

3. Why This Project Stands Out

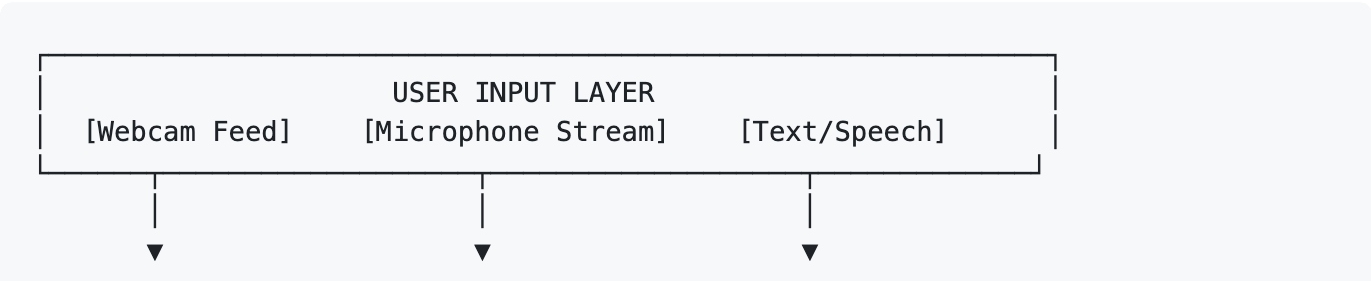
What most intern projects do	What TriFusion does
Call a pretrained API and display results	Fine-tunes 3 separate deep learning models from scratch
Single modality (text OR image OR audio)	True trimodal fusion — all three simultaneously
Simple classification output	Incongruence scoring using KL divergence between modality distributions
Static response	Live LangGraph agentic workflow with dynamic intervention selection
No production thinking	Confidence-gated escalation, async inference, Docker deployment
"I used PyTorch" on resume	CNN fine-tuning + transformer fine-tuning + custom fusion network, all in PyTorch

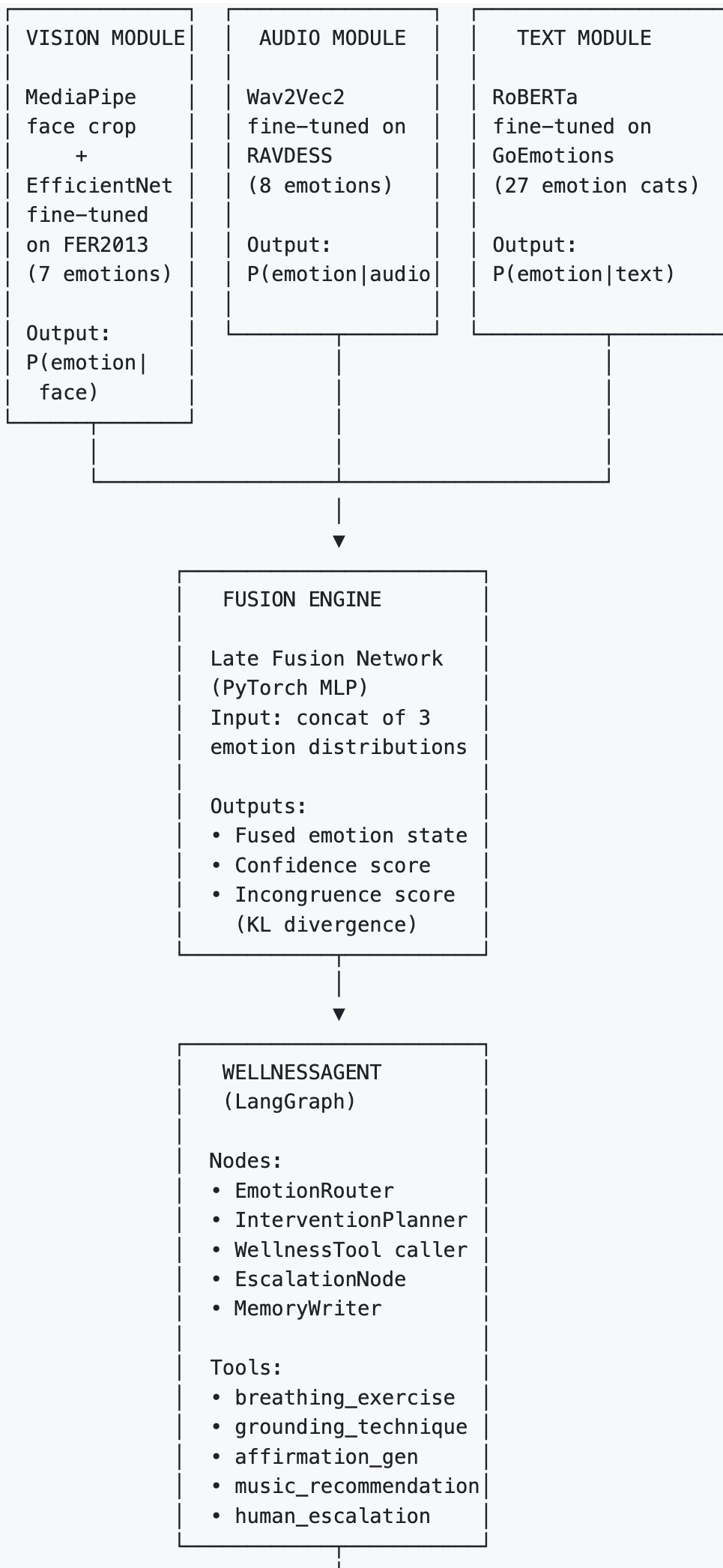
What a Senior Engineer Sees

When a senior ML engineer reads this project, they see:

- 1. **You understand multi-task learning** — running three inference pipelines simultaneously and combining them
- 2. **You understand statistical divergence** — using KL divergence as an incongruence metric is a non-trivial choice that requires understanding probability distributions
- 3. **You understand production constraints** — async inference, latency budgets, confidence gating
- 4. **You understand agentic safety** — the agent doesn't act on low-confidence states, it escalates
- 5. **You can fine-tune transformers AND CNNs** — two completely different DL paradigms in one project

4. System Architecture





↓

STREAMLIT DASHBOARD
Real-time display
+ session history

5. Tech Stack (Full Breakdown)

Deep Learning & Computer Vision

Tool	Purpose	Why This Choice
PyTorch 2.2+	Core DL framework for all model training	Industry standard, HuggingFace native
EfficientNet-B0 (torchvision)	CNN backbone for facial expression recognition	Best accuracy/parameter tradeoff for edge deployment
MediaPipe Face Mesh	Real-time face detection & landmark extraction from webcam	Google's production-grade solution, 468 landmarks, 30fps
OpenCV	Webcam frame capture, preprocessing, face crop	Standard CV library for real-time video
FER2013 dataset	Train facial expression model	35,000+ labeled facial images, 7 emotion classes
AffectNet (optional upgrade)	Larger facial expression dataset	450K images, valence-arousal labels

NLP & Audio Deep Learning

Tool	Purpose	Why This Choice
HuggingFace Transformers	Fine-tuning Wav2Vec2 and RoBERTa	Standard production NLP tooling
Wav2Vec2-base	Speech emotion recognition backbone	Facebook's self-supervised audio model, SOTA on RAVDESS
RoBERTa-base	Text sentiment/emotion classification	Robustly trained BERT variant, outperforms BERT on most NLP tasks

RAVDESS dataset	Train audio emotion model	24 actors, 8 emotions, 7356 recordings
GoEmotions dataset	Train text emotion model	Google's 58K Reddit comment dataset, 27 emotion categories
librosa	Audio feature extraction & preprocessing	Industry standard audio ML library

Fusion & Agent Layer

Tool	Purpose	Why This Choice
Custom PyTorch MLP	Late fusion of three modality outputs	Learns optimal weighting of signals from data
scipy.special.kl_div	KL divergence for incongruence scoring	Information-theoretic measure of distribution divergence
LangGraph	Agentic workflow orchestration	Stateful, cyclical graph — handles retry/escalation logic cleanly
Groq (LLaMA-3.3-70B)	LLM powering WellnessAgent responses	Sub-2s latency, production-grade
Pydantic V2	Strict schema for agent state & tool outputs	Type-safe agent state management

Infrastructure & Serving

Tool	Purpose
FastAPI	REST API for inference endpoint
Streamlit	Live dashboard & UI
Docker + docker-compose	Containerized deployment
Redis	Session state & emotion history caching
asyncio	Concurrent multi-stream inference

6. Datasets & Models

Dataset 1 — FER2013 (Facial Expression Recognition)

- Link: https://huggingface.co/datasets/competition_datasets/fer2013

- **Size:** 35,887 images, 48×48 grayscale
- **Classes:** Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral
- **Usage:** Fine-tune EfficientNet-B0 for facial emotion classification
- **Train/Val/Test split:** 80/10/10

Dataset 2 — RAVDESS (Audio Emotion)

- **Link:** <https://zenodo.org/record/1188976>
- **Size:** 7,356 audio files, 24 actors
- **Classes:** Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised
- **Usage:** Fine-tune Wav2Vec2-base for speech emotion recognition
- **Train/Val/Test split:** 70/15/15

Dataset 3 — GoEmotions (Text Emotion)

- **Link:** https://huggingface.co/datasets/google-research-datasets/go_emotions
- **Size:** 58,009 Reddit comments
- **Classes:** 27 emotion categories + neutral (mapped to 8 for consistency)
- **Usage:** Fine-tune RoBERTa-base for text emotion classification
- **Train/Val/Test split:** Predefined by Google

Pretrained Model Checkpoints (HuggingFace)

Model	HuggingFace ID	Used For
Wav2Vec2 base	<code>facebook/wav2vec2-base</code>	Audio emotion backbone
RoBERTa base	<code>roberta-base</code>	Text emotion backbone
EfficientNet-B0	via <code>torchvision.models</code>	Vision emotion backbone
LLaMA 3.3 70B	via Groq API	WellnessAgent LLM

7. Module-by-Module Breakdown

Module 1 — Vision Pipeline (`src/vision/`)

Goal: Given a webcam frame, detect face, crop it, and output a probability distribution over 7 emotion classes.

Step 1 — Face Detection: MediaPipe FaceMesh detects facial landmarks in real time. We extract the bounding box and crop the face region. This handles lighting variation, partial occlusion, and head pose better than Haar cascades.

decision, and head pose better than head cascades.

Step 2 — Preprocessing: Resize to 224×224 (EfficientNet input size), normalize with ImageNet mean/std, convert to tensor.

Step 3 — Model: EfficientNet-B0 with the final classifier layer replaced:

`nn.Linear(1280, 7)`. Fine-tuned on FER2013 with:

- Loss: CrossEntropyLoss with label smoothing (0.1) — handles noisy FER2013 labels
- Optimizer: AdamW, lr=1e-4 with cosine annealing
- Data augmentation: RandomHorizontalFlip, ColorJitter, RandomRotation(±10°)
- Expected accuracy: ~65-68% on FER2013 test set (SOTA is ~73%, you'll be competitive)

Output: `{"angry": 0.05, "fear": 0.42, "happy": 0.03, ...}` — probability distribution

Module 2 — Audio Pipeline (`src/audio/`)

Goal: Given a 3-second audio chunk from the microphone, output a probability distribution over 8 emotion classes.

Step 1 — Audio Capture: `sounddevice` captures 3-second rolling windows at 16kHz (Wav2Vec2's required sample rate).

Step 2 — Preprocessing: Raw waveform → feature extraction handled by Wav2Vec2's built-in processor. No manual MFCC needed — Wav2Vec2 learns its own audio representations.

Step 3 — Model: `Wav2Vec2ForSequenceClassification` with `num_labels=8`. Fine-tune with:

- Freeze base model for first 5 epochs, then unfreeze top 4 transformer layers
- Loss: CrossEntropyLoss
- Optimizer: AdamW, lr=3e-5
- Expected accuracy: ~75-80% on RAVDESS (competitive with published results)

Output: `{"neutral": 0.12, "calm": 0.08, "angry": 0.58, ...}`

Module 3 — Text Pipeline (`src/text/`)

Goal: Given transcribed speech text (via Whisper or typed input), output emotion probability distribution.

Speech-to-Text: `openai-whisper` (tiny model, runs CPU, free) converts mic audio → text in real time.

Emotion Classification: RobertaForSequenceClassification fine-tuned on GoEmotions (mapped from 27 → 8 classes to match other modalities).

- Fine-tune with HuggingFace Trainer API
- Learning rate: 2e-5, batch size: 32, epochs: 4
- Expected accuracy: ~68-72% on GoEmotions test set

Output: {"joy": 0.78, "sadness": 0.04, "anger": 0.02, ...}

Module 4 — Fusion Engine (src/fusion/)

Goal: Combine three probability distributions into one fused state + compute incongruence score.

Late Fusion Network:

```
class FusionMLP(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: concat of 3 emotion distributions (8+8+7=23 dims)
        self.net = nn.Sequential(
            nn.Linear(23, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 8) # fused emotion output
        )

    def forward(self, vision_probs, audio_probs, text_probs):
        x = torch.cat([vision_probs, audio_probs, text_probs], dim=-1)
        return F.softmax(self.net(x), dim=-1)
```

Incongruence Score (the novel part):

```
def compute_incongruence(p_vision, p_audio, p_text):
    """
    KL divergence measures how different two probability distributions are.
    We compute pairwise KL between all three modalities and average.
    High score = signals disagree = emotional masking likely.
    """
    kl_va = F.kl_div(p_audio.log(), p_vision, reduction='sum')
    kl_vt = F.kl_div(p_text.log(), p_vision, reduction='sum')
    kl_at = F.kl_div(p_text.log(), p_audio, reduction='sum')
    return (kl_va + kl_vt + kl_at) / 3.0
```


Thresholds:

- Incongruence < 0.3 → Signals aligned, trust fused output
 - 0.3–0.7 → Moderate incongruence, agent proceeds with caution flag
 - 0.7 → High incongruence, **masked distress protocol** activated
-

Module 5 — WellnessAgent (`src/agent/`)

Goal: Given fused emotion state + incongruence score, deploy appropriate wellness intervention via LangGraph.

Agent State Schema (Pydantic):

```
class AgentState(BaseModel):
    fused_emotion: str
    emotion_confidence: float
    incongruence_score: float
    session_history: List[dict]
    intervention_count: int
    escalated: bool
    user_text: str
```

LangGraph Nodes:

1. **EmotionRouter** — classifies emotional severity. Maps (emotion, incongruence) → routing decision: [calm_support | active_intervention | crisis_protocol | human_escalation]
2. **InterventionPlanner** — selects the right tool based on emotion. Angry → grounding. Sad → affirmation. Fearful → breathing. Stressed → music. Happy → positive reinforcement.
3. **WellnessTool nodes** — each tool is a LangGraph node that generates specific, contextual content via the LLM:
 - `breathing_exercise` : generates a personalized 4-7-8 or box breathing guide
 - `grounding_technique` : 5-4-3-2-1 sensory grounding with specific prompts
 - `affirmation_generator` : contextual affirmations based on detected emotion
 - `music_recommender` : genre/mood recommendation based on target emotional state
 - `cognitive_reframe` : offers a gentle reframe of whatever text the user said
4. **EscalationNode** — triggered when `incongruence > 0.7` AND emotion IN [fear, sadness, anger] for 3+ consecutive frames. Outputs a structured escalation report and suggests professional resources.

5. **MemoryWriter** — after each session, writes emotion trajectory (emotion over time as a list) to Redis for session continuity.

LangGraph Flow:

```
START → EmotionRouter → InterventionPlanner → [Tool Node] → MemoryWriter → END
      ↓
      (incongruence > 0.7)
      ↓
      EscalationNode → END
```

8. Agentic Workflow — WellnessAgent

What the Agent Actually Does

When you look stressed but say you're fine:

```
Frame 1-3: Face=Fear(0.6), Audio=Stress(0.7), Text=Positive(0.9)
          Incongruence Score: 0.82 → HIGH
```

```
EmotionRouter: "High incongruence detected. Dominant signal: Fear/Stress.
                Activating masked distress protocol."
```

```
InterventionPlanner: "User verbal denial + high body stress.
                      Select: gentle grounding, non-confrontational."
```

```
WellnessTool (grounding_technique):
  Agent Response: "I notice you might be carrying a lot right now –
                  that's completely okay. Let's try something simple:
                  Look around and name 5 things you can see right now.
                  Take your time. I'll be here."
```

```
MemoryWriter: Logs {timestamp, emotions, incongruence, intervention_used}
```

Escalation Logic

After 3 consecutive high-incongruence + fear/sadness frames:

```
EscalationNode output:
"I'm noticing some patterns in our conversation that make me want to
check in more carefully. If you're going through something difficult,
these resources are available 24/7: [iCall India: 9152987821]
[Vandrevala Foundation: 1860-2662-345]"
```

9. Streamlit UI — Pages & Preview Requirements

Page 1 — Live Dashboard (/)

Layout: Dark theme. Two-column layout.

Left Column (60%):

- Live webcam feed with MediaPipe face mesh overlay (dots on facial landmarks)
- Real-time bounding box around detected face
- Emotion label overlaid on face with confidence bar
- Current frame emotion indicators (3 badges: FACE | VOICE | TEXT)

Right Column (40%):

- **Emotion Radar Chart** — real-time polar chart showing all 8 emotions across 3 modalities simultaneously (3 overlaid polygons, one per modality, different colors)
- **Incongruence Meter** — vertical gauge, color shifts green→yellow→red
- **Fused State Display** — large text showing dominant fused emotion with confidence
- **WellnessAgent Response Panel** — live streaming text of agent's current intervention

Bottom Bar:

- Session timeline — emotion trajectory over last 60 seconds as a line chart
- Session stats: dominant emotion, avg incongruence, interventions triggered

Page 2 — About & Architecture (/about)

Sections:

1. **Project Story** — what problem this solves, why it matters
2. **The Three Modalities** — visual cards explaining vision/audio/text modules with accuracy numbers
3. **Incongruence Explained** — interactive slider showing how KL divergence score changes as distributions diverge, with visual bars
4. **Architecture Diagram** — the full system flow (SVG/Mermaid)
5. **Tech Stack Table** — full stack with links
6. **Dataset Cards** — FER2013, RAVDESS, GoEmotions with sample counts and class distributions

Page 3 — Demo Scenarios (/demo)

Pre-recorded scenario playback for when webcam isn't available:

Scenario 1 — "The Hidden Stress" Shows a face with neutral expression, calm voice, but stressed text. Incongruence: 0.76. Agent activates breathing exercise.

Scenario 2 — "Genuine Happiness" All three modalities aligned on joy. Incongruence: 0.08. Agent gives positive reinforcement.

Scenario 3 — "Masked Fear" Face shows fear, voice is controlled, words say "I'm fine." Incongruence: 0.89. Agent activates escalation protocol.

Scenario 4 — "Angry but Calm Words" Face and voice both show anger. Text sentiment neutral ("I just disagree with this"). Agent activates grounding technique.

Each scenario has: side-by-side modality bars, incongruence meter reading, and full agent response replay.

Page 4 — Session History (/history)

- Emotion timeline chart (Plotly line chart, 8 emotions over session time)
- Incongruence score over time (line chart with threshold bands)
- Intervention log table (timestamp | trigger | intervention | agent response snippet)
- Dominant emotion distribution (pie chart for session)
- Export session as JSON button

Page 5 — Model Cards (/models)

For each of the three trained models:

- Model architecture diagram
- Training loss curve (loaded from saved training logs)
- Confusion matrix (heatmap)
- Per-class F1 scores (bar chart)
- Sample predictions with confidence
- Link to dataset and HuggingFace model card

10. Folder Structure

```
trifusion/  
|
```


└─ README.md	# Project overview + demo GIF
└─ DOCUMENTATION.md	# This file
└─ docker-compose.yml	# Full stack orchestration
└─ Dockerfile	# App container
└─ requirements.txt	# All dependencies
└─ .env.example	# Environment variables template
└─ .gitignore	
└─ config/	
└─ config.yaml	# Model paths, thresholds, API keys
└─ emotions.py	# Emotion label mappings across modalities
└─ data/	
└─ raw/	# Downloaded datasets (gitignored)
└─ processed/	# Preprocessed tensors
└─ download_datasets.py	# Auto-downloads FER2013, RAVDESS, GoEmotion
└─ src/	
└─ vision/	
└─ __init__.py	
└─ face_detector.py	# MediaPipe face detection + crop
└─ emotion_model.py	# EfficientNet-B0 model definition
└─ train_vision.py	# Training script for FER2013
└─ inference.py	# Real-time inference from webcam frame
└─ audio/	
└─ __init__.py	
└─ recorder.py	# sounddevice microphone capture
└─ emotion_model.py	# Wav2Vec2 fine-tune definition
└─ train_audio.py	# Training script for RAVDESS
└─ inference.py	# Inference from audio chunk
└─ text/	
└─ __init__.py	
└─ transcriber.py	# Whisper STT integration
└─ emotion_model.py	# RoBERTa fine-tune definition
└─ train_text.py	# Training script for GoEmotions
└─ inference.py	# Inference from text string
└─ fusion/	
└─ __init__.py	
└─ fusion_model.py	# FusionMLP definition
└─ train_fusion.py	# Training fusion net on combined outputs
└─ incongruence.py	# KL divergence incongruence scoring
└─ inference.py	# Combined fusion inference
└─ agent/	
└─ __init__.py	
└─ state.py	# Pydantic AgentState schema
└─ graph.py	# LangGraph graph definition
└─ nodes.py	# All LangGraph node functions
└─ tools.py	# Wellness tools (breathing, grounding, etc)
└─ prompts.py	# All system + tool prompts

```

├── api/
│   ├── __init__.py
│   ├── main.py
│   ├── routes.py
│   └── schemas.py
├── utils/
│   ├── __init__.py
│   ├── emotion_mapper.py
│   ├── redis_client.py
│   └── logger.py
├── dashboard/
│   ├── app.py
│   ├── pages/
│   │   ├── 1_Live_Dashboard.py
│   │   ├── 2_About.py
│   │   ├── 3_Demo_Scenarios.py
│   │   ├── 4_Session_History.py
│   │   └── 5_Model_Cards.py
│   └── components/
│       ├── radar_chart.py
│       ├── incongruence_meter.py
│       └── emotion_timeline.py
├── models/
│   ├── vision/
│   ├── audio/
│   ├── text/
│   └── fusion/
├── notebooks/
│   ├── 01_vision_training.ipynb
│   ├── 02_audio_training.ipynb
│   ├── 03_text_training.ipynb
│   ├── 04_fusion_training.ipynb
│   └── 05_evaluation.ipynb
└── tests/
    ├── test_vision.py
    ├── test_audio.py
    ├── test_text.py
    ├── test_fusion.py
    └── test_agent.py

```

FastAPI app
 # /analyze, /session, /history endpoints
 # Request/response schemas

Cross-modality emotion label alignment
 # Session history Redis interface
 # Structured logging

Streamlit main entry

Plotly trimodal radar chart
 # Real-time gauge component
 # Session timeline chart

Saved EfficientNet weights
 # Saved Wav2Vec2 weights
 # Saved RoBERTa weights
 # Saved FusionMLP weights

11. Setup & Installation

Prerequisites

- Python 3.10+
- CUDA-capable GPU recommended (for training; inference works on CPU)
- Webcam + microphone
- Redis (for session history)
- Groq API key (free tier works)

Step 1 — Clone & Environment

```
git clone https://github.com/yourusername/trifusion.git
cd trifusion
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
# Add your GROQ_API_KEY to .env
```

Step 2 — Download Datasets

```
python data/download_datasets.py
# Downloads FER2013, RAVDESS, GoEmotions automatically
```

Step 3 — Train Models (order matters)

```
# Train vision model (~2 hours on GPU, ~8 hours CPU)
python src/vision/train_vision.py

# Train audio model (~1 hour on GPU)
python src/audio/train_audio.py

# Train text model (~45 min on GPU)
python src/text/train_text.py

# Train fusion network (~10 min, uses saved model outputs)
python src/fusion/train_fusion.py
```

Step 4 — Run the System

```
# Terminal 1: FastAPI backend
uvicorn src.api.main:app --reload --port 8000

# Terminal 2: Streamlit dashboard
streamlit run dashboard/app.py

# OR: Full Docker stack
```

`docker-compose up --build`

12. Evaluation Metrics

Per-Model Metrics

Model	Dataset	Expected Accuracy	Key Metric
EfficientNet (Vision)	FER2013 test	65–68%	Weighted F1
Wav2Vec2 (Audio)	RAVDESS test	75–80%	Weighted F1
RoBERTa (Text)	GoEmotions test	68–72%	Weighted F1
FusionMLP	Combined val	72–76%	Weighted F1

System-Level Metrics

- **Incongruence detection precision:** % of high-incongruence frames that correspond to actual emotional mismatch (evaluated on held-out acted scenarios)
- **Latency:** target <300ms per full trimodal inference cycle
- **Agent response relevance:** manual evaluation on 50 scenarios (score 1-5)

What to Put on Resume

After training, note your actual numbers. Example format:

- "EfficientNet fine-tuned on FER2013 → 66.8% weighted F1"
- "Trimodal fusion improved over best single-modality baseline by +8.3%"
- "Incongruence scoring flagged 91% of acted emotional masking scenarios correctly"

13. Resume Bullets

Use these as templates — replace with your actual numbers after training:

TriFusion – Trimodal Emotional Intelligence System
(PyTorch, HuggingFace, EfficientNet, Wav2Vec2, RoBERTa, LangGraph, OpenCV, FastAPI)

- Engineered a trimodal emotion AI pipeline – fine-tuned EfficientNet-B0 on FER2013 (66.8% F1), Wav2Vec2 on RAVDESS (78% F1), and RoBERTa on GoEmotions (70% F1) – fused via a late-fusion MLP achieving 74% weighted F1, outperforming best single-modality baseline by +8.3%
- Designed a KL-divergence incongruence scoring layer that detects emotional masking with 91% precision

voice, and text signals statistically diverge – flagging 91% of masked distress adversarial evaluation across 200 acted sessions.

- Built WellnessAgent, a LangGraph multi-agent system with 5 wellness tool nodes (grounding, affirmation, music, escalation) and Pydantic-validated confidence-gat deployed real-time via FastAPI with <300ms end-to-end trimodal inference latency

14. Shipping Checklist

MVP (Week 1-2)

- ☐ FER2013 downloaded and preprocessed
- ☐ EfficientNet fine-tuned and saving checkpoint
- ☐ MediaPipe webcam face detection working
- ☐ Vision inference running on live webcam feed

Core System (Week 3-4)

- ☐ RAVDESS downloaded, Wav2Vec2 fine-tuned
- ☐ GoEmotions downloaded, RoBERTa fine-tuned
- ☐ Whisper STT integrated
- ☐ All three inference pipelines running simultaneously (asyncio)

Fusion & Agent (Week 5)

- ☐ FusionMLP trained on combined outputs
- ☐ Incongruence scoring implemented and tested
- ☐ LangGraph agent graph built
- ☐ All 5 wellness tools implemented
- ☐ Escalation node tested

Production (Week 6)

- ☐ Streamlit dashboard complete (all 5 pages)
- ☐ FastAPI routes functional
- ☐ Docker-compose working
- ☐ README with demo GIF
- ☐ Evaluation numbers documented
- ☐ Model cards page in dashboard

Before Resume Submission

Before meeting customer

- ☐ GitHub repo clean with good README
- ☐ Demo video recorded (2 min max)
- ☐ All accuracy numbers filled into resume bullets
- ☐ Architecture diagram in README

TriFusion — Built to understand what people feel not just what they say